iPRA with Computation in Term Syntax

A six-rule presentation, a completeness repair, and an exact proof-checker comparison

Prepared for Vladimir Reshetnikov with ChatGPT

11 November 2026

Abstract

Both equations of scalar iteration can be made definitional by using full computation normal forms and hereditary substitution. The resulting presentation of full first-order intuitionistic primitive recursive arithmetic has six inference patterns: hypothesis, reflexivity, crossed one-term equality transport, closure elimination, witnessed-closure elimination, and one introduction rule. There is no separate computation-equation axiom. We prove strong normalization, confluence, coherent substitution, exact consequence equivalence with the existing native calculus, and both interpretation composites.

The main new completeness argument replaces a numerical zero alias with a compound formula envelope. It recovers the atomic case-introduction instance that would otherwise be lost when normalization changes a witness to literal zero. This proof does not require a nonliteral term equal to zero.

A complete tagged checker uses 2,529 normalized Python bytes, against 2,532 for a matched zero-only-normalizing control and 2,569 for a matched raw control. The 3-byte improvement over the closest control is narrow: token and AST counts increase, and the repository's separate compact arity codec remains much smaller. This is a local presentation and source-byte reduction, not a proof of an absolutely simplest theory or a simultaneous improvement of every complexity coordinate. Uniform normalization is total recursive but not primitive recursive; its code is included in the measured kernel. The new universal results are written mathematical proofs, supported by executable translations and finite tests, rather than a new Lean or Rocq formalization.

Contents

1 Result and comparison boundary **4**
1.1 What is smaller . . . 4
1.2 Novelty within the inspected snapshot . . . 5

2 The complete normalized calculus and the envelope repair **5**
2.1 Six primitive inference patterns . . . 5
2.2 The compound envelope . . . 6
2.3 Exact comparison and the interpretations . . . 7

3 Normalization and the comparison with the raw calculus **8**
3.1 Strong normalization and unique normal forms . . . 8
3.2 Hereditary substitution and the shared walk . . . 9
3.3 Equality inside an update binder . . . 9
3.4 Both consequence translations and both composites . . . 10
3.5 The uniform cost is external . . . 11

4 The full iPRA interpretation, explicitly **11**
4.1 Recovering ordinary intuitionistic logic . . . 11
4.2 The intermediate ordinary theory . . . 12
4.3 Arithmetic compilation and both numerical composites . . . 13

5 Implementation, evidence, and remaining limits **14**
5.1 The exact trusted boundary . . . 14
5.2 Reproducible measurements . . . 14
5.3 Executed checks . . . 15
5.4 The external normalization cost . . . 15
5.5 What remains unresolved . . . 16

Attribution and the inherited arithmetic foundations **17**

A Complementary introductions with one local assumption **17**
A.1 Complete native syntax and rule inventory . . . 17
A.2 Exact equivalence and the unused-witness construction . . . 18
A.3 Induction and numerical cases separate cleanly . . . 19
A.4 The entire ordinary logical interface . . . 20
A.5 Whole-domain realizations and both formula composites . . . 21
A.6 What the theorem does and does not claim . . . 22

B Arithmetic strength and both ordinary interpretation composites **22**
B.1 The arithmetic dependency order . . . 23

C Internal scalar predecessor bootstrap **24**

C.1 Elementary terms and two finite cycles . . . 24
C.2 A ramp with a global scaling identity . . . 25
C.3 A comparator from scalar iteration . . . 26
C.4 Saturated iteration gives predecessor . . . 26
C.5 Dependency boundary . . . 27

D From literal-equation induction to full intuitionistic PRA 28
D.1 The exact hypotheses . . . 28
D.2 Constructor separation and two elementary tests . . . 28
D.3 Four elementary uses of equation induction . . . 29
D.4 An equation invariant that reflects actual equality . . . 30
D.5 Compilation of all quantifier-free formulas . . . 31
D.6 Recovery of induction and the full theory . . . 32

E Pairing, primitive recursion, and internal numerical composites 33
E.1 Pairing from a bounded square-root search . . . 33
E.2 Compiling each primitive recursion . . . 34
E.3 Why both numerical round trips are internal . . . 34

F The selected complete checker 35

1 Result and comparison boundary

The target is full first-order intuitionistic primitive recursive arithmetic: intuitionistic first-order equality logic, a function symbol for each fixed primitive-recursive definition, its defining equations, successor separation and injectivity, and quantifier-free induction with parameters. No induction for arbitrary quantified formulas is intended.

The comparison is the repository snapshot `11be5aabc6b07262046e2553419bc6b05271d178`. The snapshot contains the raw native calculus, the seven-rule witness-selected version, two zero-normalizing six-rule variants, and many checker representations [1, 2, 9, 8]. All six currently incoming archive summaries and relevant mathematical specifications were also inspected [10].

Theorem 1.1 (Main result)**.** *There is a six-rule proof presentation $\mathcal{F}$ whose numerical terms are full scalar-iterator computation normal forms and whose formulas have only equation, Q, and D constructors. It has no separate iterator computation axioms. Its ordinary realization is bi-interpretable and equiconsistent with full first-order* iPRA*.*

The interpretations use the whole numerical domains and actual equality. The isomorphisms from each numerical domain to its double interpretation have graph $x = y$. Both formula round trips are provably equivalent, not merely equivalent in the standard model.

The proof has three distinct parts. First, both iterator reductions terminate and yield coherent canonical substitution. Second, a compound envelope repairs the only missing case of the witness-selected introduction rule. Third, exact sequent translations connect the new presentation with the inherited raw native calculus, so that its full iPRA bridge transfers. The arithmetic foundations are reproduced with attribution in the appendices.

1.1 What is smaller

The mathematical change moves both computation schemas into a terminating term-construction operation. Reflexivity replaces the explicit computation leaves. Relative to the raw seven-rule presentation this removes one inference pattern; relative to the incoming six-rule zero-normal form, the number of patterns remains six, but the residual successor-computation schema is replaced by the simpler reflexivity schema. The normalizer is trusted code and is counted in every reported size.

The matched complete checkers use the same tagged syntax, goal-directed protocol, typed traversal, scope checks, JSON interface, and error policy. The only differences are which iterator computations the traversal normalizes and which equations can close an axiom leaf.

Complete standalone program	Bytes	Tokens	AST	Gzip-9
Raw computations, matched control	2569	895	930	941
Zero-only normalization, matched control	2532	890	930	945
Full normalization, block cases	2546	904	962	960
Full normalization, expression cases	**2529**	908	965	963

The selected expression-form checker saves 40 bytes against the matched raw control and 3 bytes against the matched zero-only control. The latter is $3/2532 \approx 0.1185\%$. It has 18 more significant tokens and 35 more AST nodes than that closest control. Gzip increases by 18 bytes and is reported only for information. The expression form spells the same three normalization cases in one conditional expression; it is not a different calculus.

The earlier strict forward zero-normal checker has 3095 normalized bytes. The new auxiliary forward checker has 3100, so normalization does *not* improve that implementation. Most of the reduction from

older tagged forward checkers to the new goal-directed family comes from sharing validation with substitution and from inferring leaves from the goal. It would be misleading to attribute that entire change to the new mathematical idea.

Limits of the complexity claim. The repository's separate 516-byte arity-coded checker remains smaller, under its different representation and public-input contract. No global byte, token, or AST record is claimed. Existing six-rule presentations are acknowledged. The present result gives a genuine change in the computational axiom interface and a narrow matched byte improvement, but does not settle the request for an absolutely simplest axiomatization or establish a strict improvement in every stated complexity coordinate.

1.2 Novelty within the inspected snapshot

Zero normalization and canonical successor-seeded recursors already remove one explicit computation equation [9, 8]. The incoming initializer and template-free reports simplify equality instructions, and the diagonal guard changes how introduction modes are selected. The construction here normalizes *both* scalar computations, retains the original meanings of Q and D, and repairs the missing atomic case through a compound logical envelope instead of a nonliteral numerical zero. This is the distinguishing contribution relative to the inspected packages. A literature-wide priority claim is not made.

2 The complete normalized calculus and the envelope repair

Let nf be the unique computation normal form established below. Canonical numerical terms are the normal forms of

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Thus every iterator counter has neither zero nor successor as its root; all its children are canonical. The update variable u is bound only in h. Put $1 = S0$. Formula roots are retained:

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

For every displayed instance, substitution means capture-free substitution followed by normalization. In particular, an expression $I(n, a; u.h)$ denotes the normal form of the displayed constructor expression, even when substitution exposes a computation. Formula normalization changes numerical subterms and never removes a Q or D.

The intended readings, extended recursively to matrices, are

$$Q_y B(y) \;:=\; B(0) \to \forall y\, B(y), \qquad D_{x,y} B(x, y) \;:=\; \exists x \big(B(x, 0) \to \forall y\, B(x, y)\big).$$

The designated contradiction is $0 = S0$. Contexts are finite reusable lists.

2.1 Six primitive inference patterns

The calculus $\mathcal{F}$ has exactly the following primitive patterns.

1. *Hypothesis:* $\Gamma \vdash A$ if $A \in \Gamma$.
2. *Reflexivity:* $\Gamma \vdash t = t$ for every canonical term t.

3. *Crossed transport in one term:*
$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = r}{\Gamma \vdash r = h(t)}. \tag{Tr}$$
Here $h(v)$ is any fixed numerical term template, including a constant template. There is no restriction on the source s.

4. *Closure elimination:*
$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}. \tag{QE}$$

5. *Paired elimination:*
$$\frac{\Gamma \vdash D_{x,y} B(x,y) \qquad \Gamma, Q_y B(b,y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$
The eigenvariable b is fresh for the original context, conclusion, and fixed matrix parameters.

6. *Witness-directed introduction:*
$$\frac{\Gamma, B(t,\eta) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y} B(x,y)}, \qquad \eta = \begin{cases} z, & B \text{ is an equation and } t \text{ is literally } 0, \\ 0, & \text{otherwise.} \end{cases} \tag{WI}$$
Here z is fresh for the context, witness t, and fixed matrix parameters.

These are all the logical and nonlogical inference patterns. There is no additional computation axiom: both iterator equations are enforced by normalized substitution. Their implementation cost remains part of the checker. Ordinary logical connectives are subsequently recovered as in the native scalar calculus; they are not extra primitive rules.

Weakening, fresh renaming, and ordinary-assumption cut follow directly by induction on derivations. They preserve the literal-zero and formula-root tests. Numerical substitution is *not* claimed by that direct argument: it can normalize a nonzero-spelled witness to zero. It will follow from the comparison with a substitution-stable calculus below.

2.2 The compound envelope

First, general closure introduction by cases. For every matrix B, the rule
$$\frac{\Gamma, B(0) \vdash B(Sz)}{\Gamma \vdash Q_y B(y)} \tag{QC}$$
is already admissible. Introduce $D_{x,y} B(y)$ with an unused witness coordinate x and witness 1. Since 1 is not literally zero, WI uses its case branch regardless of the root of B. DE and its local hypothesis then export $Q_y B(y)$. This construction does not use any omitted atomic case-introduction instance at witness zero.

An equation has a compound representative. For E the equation $s = t$, define
$$G(v,E) := \big(I(v,0;u.s) = I(v,0;u.t)\big), \qquad L(E) := Q_v G(v,E), \tag{Envelope}$$
where u, v are fresh; the endpoints are lifted under these binders. Computation gives the literal normalized identities
$$G(0,E) \equiv (0=0), \qquad G(Sw,E) \equiv E.$$
Consequently, under every retained context,
$$E \vdash L(E), \qquad L(E) \vdash E. \tag{Envelope equivalence}$$

For the first direction apply QC to the gate matrix: under its local assumption $0 = 0$, the successor instance is the retained equation E. For the reverse direction use QE at 1 and reflexivity $0 = 0$. The envelope has a Q root after normalization. These derivations are uniform in all parameters of E and use neither arithmetic induction nor a numerical case rule.

Recovering the missing atomic case. Suppose a supplied derivation proves

$$\pi : \quad \Gamma, E(0,0) \vdash E(0, Sz), \qquad E(x,y) \text{ an equation.}$$

Set $F(x,y) = L(E(x,y))$. Under $F(0,0)$, the reverse envelope direction supplies $E(0,0)$. Cut with π and then use the forward envelope direction to obtain $F(0,Sz)$. WI at witness zero now uses *cases*, because F has a compound root, and proves $D_{x,y}F(x,y)$.

Eliminate this formula at a fresh b, retaining $Q_y F(b,y)$. To introduce $D_{x,y}E(x,y)$ at witness b, assume $E(b,0)$, convert it to $F(b,0)$, use the retained closure at Sz, and convert $F(b,Sz)$ back to $E(b,Sz)$. The fresh variable b is not literally zero, so this second WI also uses cases. DE closes the derivation, yielding the required $D_{x,y}E(x,y)$. Neither the final formula nor the original context contains b.

It follows that unrestricted case introduction is admissible:

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}. \tag{D-case}$$

All instances except the repaired one are already WI. This replaces the earlier nonliteral-zero-alias argument. Full normalization erases closed zero aliases; the compound envelope works without one.

Direct equation progress. From $\Gamma, E(z) \vdash E(Sz)$, with E an equation, use an unused witness coordinate and witness zero in WI. DE then exports

$$\Gamma \vdash Q_y E(y). \tag{Q-progress}$$

No case repair is needed for this construction.

2.3 Exact comparison and the interpretations

Let $\mathcal{F}^+$ be the substitution-stable auxiliary calculus on canonical terms, with Hyp, reflexivity, Tr, QE, DE, D-case, and Q-progress. The preceding constructions derive every rule of $\mathcal{F}^+$ in $\mathcal{F}$. Conversely, a case-mode WI is D-case. For a progress-mode WI, its equation matrix and witness zero give Q-progress for $E(0,y)$; under $E(0,0)$, QE supplies $E(0,Sz)$, and D-case at zero gives the same conclusion. The other rules are common. Thus, for every canonical sequent,

$$\Gamma \vdash_{\mathcal{F}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{F}^+} A. \tag{Exact consequence}$$

Ordinary equality reasoning is available from reflexivity and Tr. Symmetry follows by taking the major premise $s = s$ and minor premise $s = t$ with the identity template. Congruence follows by taking a reflexive minor $h(s) = h(s)$ along $s = t$. Two crossed transports give transport in an arbitrary equation template. Transport in compound formula templates follows by external induction on their roots: for Q, use QC, transport its base backwards, apply the retained closure, and transport its successor conclusion forwards; for D, eliminate the old formula and apply D-case at the resulting witness with the same two conversions. This is a syntactic induction on formulas, not an additional arithmetic-induction schema.

3 Normalization and the comparison with the raw calculus

This section supplies the metatheory of the computation built into the new syntax. Write $\mathcal{N}$ for the raw native calculus with its two computation axioms, and $\mathcal{F}$ for the six-rule normalized calculus. All statements about normalization are external statements about finite syntax; they are not new axioms of the numerical theory.

3.1 Strong normalization and unique normal forms

Definition 3.1 (Contextual computation)**.** On raw terms, close the two reductions

$$\begin{aligned} I(0, a; u.h) &\longrightarrow a, \\ I(Sn, a; u.h) &\longrightarrow h[I(n, a; u.h)/u] \end{aligned} \tag{1}$$

under every term context, including iterator-update contexts. Substitution avoids capture. A term is *strongly normalizing* if it admits no infinite reduction sequence. For such a term t, let $\nu(t)$ be its maximum reduction length, and let $|t|$ be its syntax-tree size.

The maximum exists: a finite term has only finitely many one-step reducts, and a finitely branching well-founded reduction tree has finite height.

Lemma 3.2 (Iterator closure)**.** *Suppose n and a are strongly normalizing, and $h[v/u]$ is strongly normalizing for every strongly normalizing term v. Then $I(n, a; u.h)$ is strongly normalizing.*

Proof. The body h itself is strongly normalizing, by taking $v = u$. Use lexicographic induction on

$$\big(\nu(n),\ |n|,\ \nu(a) + \nu(h)\big).$$

It suffices to show that every one-step reduct of the iterator is strongly normalizing. A reduction in n decreases the first coordinate. A reduction in a or h decreases the last coordinate. In the latter case, if $h \longrightarrow h'$, then $h'[v/u]$ is a reduct of $h[v/u]$ for every v, so the required universal hypothesis persists. Here we use the immediate fact that contextual computation is stable under capture-free substitution.

The zero root reduct is a. At a successor root, write $n = Sp$. Then $\nu(p) = \nu(n)$ and $|p| < |n|$. The induction hypothesis therefore makes $I(p, a; u.h)$ strongly normalizing. The hypothesis on h makes $h[I(p, a; u.h)/u]$ strongly normalizing, as required. □

Theorem 3.3 (Strong normalization)**.** *Substitution of strongly normalizing terms for the free variables of any finite raw term produces a strongly normalizing term. In particular, every finite raw term is strongly normalizing.*

Proof. Induct on the original term. The variable, zero, and successor cases are immediate. At an iterator, the induction hypotheses for its counter and initializer give strongly normalizing instances. The body induction hypothesis applies to *every* additional assignment $u := v$ with v strongly normalizing. It supplies exactly the universal premise of Lemma 3.2. Freshen the bound variable before making these assignments. Finally substitute each free variable with itself. □

Lemma 3.4 (Confluence and canonical syntax)**.** *Every raw term t has a unique normal form $\mathrm{nf}(t)$, up to renaming of bound variables, or literally when de Bruijn indices are used. Its iterator subterms have normal children and counters headed by neither 0 nor S.*

Proof. The reduction is locally confluent. Disjoint redexes commute. The two root patterns in (1) are disjoint. For a redex nested inside a root contraction, contract all its residual copies after the root step. The zero rule may erase the update. The successor rule may copy the update and recursive iterator into several occurrences of the bound variable, but there are only finitely many copies; substitution stability gives a common reduct. These exhaust the overlaps.

Strong normalization and local confluence give confluence by induction on reduction height. A term is normal precisely when its children are normal and neither root iterator rule applies, giving the stated grammar. □

3.2 Hereditary substitution and the shared walk

For a substitution σ, write

$$t\langle\langle\sigma\rangle\rangle := \mathrm{nf}(t[\sigma]).$$

Formula normalization preserves the constructors $=$, Q, and D, and normalizes all their numerical terms, including terms under binders.

Lemma 3.5 (Substitution coherence)**.** *For every raw term t and substitution σ,*

$$\mathrm{nf}(t[\sigma]) = \mathrm{nf}\big(\mathrm{nf}(t)[x \mapsto \mathrm{nf}(\sigma x)]\big). \tag{2}$$

Consequently normalized substitution has the identity, composition, weakening, and binder-lifting laws. The same assertions hold for formulas.

Proof. The reduction $t \longrightarrow^* \mathrm{nf}(t)$ survives raw substitution. Reducing each of the finitely many substituted copies of σx to its normal form gives conversion to the right-hand input in (2). Uniqueness of normal forms proves the equation. Apply it twice and use associativity of raw substitution to get

$$(t\langle\langle\sigma\rangle\rangle)\langle\langle\tau\rangle\rangle = t\langle\langle x \mapsto (\sigma x)\langle\langle\tau\rangle\rangle\rangle\rangle.$$

Renamings and lifted substitutions are special cases. Formula constructors have no computation rules, so structural recursion gives the formula claims. □

The executable walk first processes the children of a node, then contracts an exposed iterator redex. At a successor it passes the raw recursive iterator into hereditary substitution; when the bound variable actually occurs, the substitution's shifting walk normalizes that replacement. If the update ignores its variable, the recursive iterator is erased without evaluation. Thus every computational step contracts a genuine redex in the materialized raw substituted term. Between contractions the algorithm traverses finite subtrees and renames indices. This is ordinary normalization with raw substitution fused into traversal: strong normalization proves termination, and the normal, reachable result is $\mathrm{nf}(t)$ by uniqueness. The same argument identifies the fused substitution with $\mathrm{nf}(t[\sigma])$.

3.3 Equality inside an update binder

There is a small but essential proof obligation when comparing the new syntax with $\mathcal{N}$. Capture-free one-term transport alone does not allow a formerly free variable in its equality premise to become bound inside an iterator. The missing congruence is derived using equation induction, which $\mathcal{N}$ already proves.

Lemma 3.6 (Iterator-body extensionality)**.** *If u is fresh for Γ and $\Gamma \vdash_{\mathcal{N}} h(u) = k(u)$, then, for all terms n, a,*

$$\Gamma \vdash_{\mathcal{N}} I(n, a; u.h) = I(n, a; u.k).$$

Proof. Choose z fresh for all passive parameters and induct on the literal equation

$$E(z) := I(z, a; u.h) = I(z, a; u.k).$$

Both zero computations give the base. Assuming $E(z)$, successor computation, numerical substitution in the pointwise equality, and ordinary equality transport give

$$\begin{aligned} I(Sz, a; h) &= h(I(z, a; h)) \\ &= k(I(z, a; h)) \\ &= k(I(z, a; k)) \\ &= I(Sz, a; k). \end{aligned}$$

Equation induction gives $E(n)$. No induction on a quantified formula or on an implication is used. □

Lemma 3.7 (Normalization is provable in the raw calculus)**.** *For each fixed finite raw term t,*

$$\vdash_{\mathcal{N}} t = \mathrm{nf}(t).$$

For each raw formula A, the formulas A and $\mathrm{nf}(A)$ are interderivable in $\mathcal{N}$, uniformly under additional retained contexts.

Proof. Each root reduction is an old computation axiom. Transport handles replacement in a successor, counter, or initializer; repeated use of Lemma 3.6 handles replacement under update binders. Thus each step of a finite normalization derivation is a provable equality. Transitivity gives the term assertion.

Induct externally on formula syntax. The equation case uses the two term equalities and ordinary equality reasoning. At a Q node, convert its new zero instance to the old zero instance, specialize the old closure at Sz, convert the result, and apply the derived case introduction for Q; reverse the conversions for the converse. At a D node, eliminate the source at a fresh witness, perform the preceding closure conversion, and introduce the target at that same witness. These constructions are uniform under passive contexts and avoid capture. They are binder congruence, not a claim that an open numerical variable may be captured by transport. □

3.4 Both consequence translations and both composites

Let T normalize every numerical term of a raw formula, and let U regard a canonical formula as raw syntax. Let $\mathcal{F}^+$ be the auxiliary normalized calculus with separate general case introduction and direct equation-progress introduction. The compound-envelope argument gives the same derivable sequents in $\mathcal{F}^+$ and $\mathcal{F}$.

Theorem 3.8 (Exact comparison)**.** *For every raw sequent and every canonical sequent,*

$$\begin{aligned} \Gamma \vdash_{\mathcal{N}} A \quad &\Longleftrightarrow \quad T\Gamma \vdash_{\mathcal{F}} TA, \\ \Delta \vdash_{\mathcal{F}} B \quad &\Longleftrightarrow \quad U\Delta \vdash_{\mathcal{N}} UB. \end{aligned}$$

Moreover $TU(B) = B$ literally, and A and $UT(A)$ are uniformly interderivable in $\mathcal{N}$.

Proof. First translate a raw derivation into $\mathcal{F}^+$ by induction on the derivation. Both old computation equations normalize to reflexivity. The remaining rule instances normalize to their corresponding instances by Lemma 3.5; literal-equation status and eigenvariable conditions are preserved. Replace auxiliary introductions by their six-rule derivations. This avoids the invalid direct argument that a nonzero raw witness must remain nonzero after normalization.

Conversely induct on an $\mathcal{F}$ derivation. Reflexivity is already derived in $\mathcal{N}$. Before applying an old rule, use Lemma 3.7 to convert its normalized instance premises to raw instances; convert its conclusion back afterwards. A case-mode witness introduction uses old case introduction. A progress-mode introduction first uses old equation progress on the instantiated literal equation, then obtains the witnessed D formula by old case introduction and closure specialization. Hypothesis, transport, Q elimination, and D elimination translate in the same way. Context conversions use the admissible assumption cut and weakening of the old calculus.

The composite TU is the identity because canonical terms are normal. The other composite is formula normalization, handled by Lemma 3.7. Combining these facts with the two forward translations proves both displayed equivalences, including the directions whose context was normalized. □

These maps interpret the term presentations modulo provable equality; normalization is not an additional numerical function symbol. Composing them with the inherited native-to-iPRA bi-interpretation gives the required formula round trips and equiconsistency. The numerical domains remain whole, equality remains actual equality, and both domain isomorphisms have graph $x = y$. Only finite *term syntax* is identified modulo computation; there is no quotient or restriction of the numerical domain.

3.5 The uniform cost is external

Every fixed scalar term denotes a primitive-recursive function, but the normalizer on *all* finite term codes is not primitive recursive. Otherwise normalizing one-variable scalar terms on numeral inputs would provide a primitive-recursive evaluator $\mathrm{Eval}(e, n)$ for all primitive-recursive functions. Primitive-recursive syntactic coding and extraction of a normal numeral's value would then make $g(e) = \mathrm{Eval}(e, e) + 1$ primitive recursive, contradicting its own scalar-term index. Invalid codes can be assigned the default value zero, so they do not affect the diagonal argument.

The size comparison therefore counts a total recursive normalizing checker, with potentially enormous running time and output. It does not assert that iPRA proves a uniform normalization theorem. If one separately requires a primitive-recursive certificate relation, finite reduction traces may be supplied and checked locally; that is a different checker whose added code must also be counted. The new normalization and comparison results above are written mathematical proofs, not claims of a new proof-assistant verification.

4 The full iPRA interpretation, explicitly

The exact consequence theorem in Section 3 reduces the arithmetic-strength question to the raw native calculus $\mathcal{N}$. This section states the connecting interpretations and the inherited arithmetic dependency order explicitly. The appendix gives the underlying proofs, rather than replacing them with standard-model calculations.

4.1 Recovering ordinary intuitionistic logic

In raw scalar syntax define the numerical selector

$$C(v, a, b) = I(v, a; u.b), \qquad C(0, a, b) = a, \quad C(Sv, a, b) = b.$$

For the normalized presentation take the normal form of each displayed term. A formula gate $G_v A$ is obtained by replacing every equation $s = t$ with $C(v, 0, s) = C(v, 0, t)$ and passing the gate through Q

and D, renaming bound variables to avoid capture. It satisfies

$$\vdash G_0 A, \qquad G_{Sv} A \dashv\vdash A.$$

The case rules and binder congruence extend a numerical selector to a formula selector $M_v(A, B)$ with

$$M_0(A, B) \dashv\vdash A, \qquad M_{Sv}(A, B) \dashv\vdash B.$$

For this selector construction, first elaborate each Q as a D with an unused witness. When the resulting formulas have unequal binder depths, insert a vacuous witnessed gate to align them. The exact structural recursion and its proof appear in the inherited native-basis appendix.

Let $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. The following are formula macros, not additional primitive logical rules:

$$\begin{aligned} \bot_* &:= (0 = S0), \\ A \Rightarrow_* B &:= Q_y M_y(A, B), \\ A \wedge_* B &:= Q_y G_y M_{p(y)}(A, B), \\ \exists_* x\, A(x) &:= D_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* v\, M_v(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* Q_y A(y). \end{aligned}$$

Every newly bound variable is fresh; normalize all numerical term expressions in the macros.

For implication, case introduction supplies a closure from the assumed zero selector to the successor selector, and QE at 1 gives elimination. For conjunction the gated base is true, and the instances $1, 2$ give the two operands because $p(1) = 1$ and $p(2) = 0$. Numerical cases give its introduction. The existential macro uses its explicit witness and a true gated base; its elimination uses DE and QE at 1. Disjunction uses selector witnesses $0, 1$ and numerical cases. Universal introduction and elimination are exactly the base instance together with its conditional closure. These derivations supply all ordinary intuitionistic natural-deduction rules.

Under $0 = 1$, equality transport through $C(v, s, t)$ gives $s = t$ for arbitrary terms. External induction on formula syntax then gives every native formula, so the designated contradiction supports explosion. No excluded-middle schema for quantified formulas is introduced.

4.2 The intermediate ordinary theory

Let T_I be intuitionistic first-order logic with actual equality and genuine falsity, the scalar terms, their two computation equations, contextual induction for literal equations, and

$$\forall x\, (x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The last sentence is an explicit falsity bridge. Positive equations alone would permit a singleton structure, so this bridge cannot be silently discarded from the ordinary realization.

The ordinary interpretation is

$$(s = t)^\circ = (s = t), \quad (Q_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y), \quad (D_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).$$

Case introduction is sound by coverage. Equation-progress introduction is sound by equation induction under the temporarily assumed base. The other rules are ordinary equality and logical reasoning. Conversely the displayed native macros, equation induction derived from Q-progress and QE, numerical

cases, and explosion reconstruct T_I. Coverage itself follows by cases, using the native disjunction and existential macros.

The key native formula composite is

$$D_{x,y}B \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)\big).$$

Forwards, DE opens a witness; under its base the exported closure gives every instance. Introduce the three macros and close DE. Backwards, open the existential at a fresh witness, assume its base, and use implication and universal elimination at Sz to introduce the original D by cases. The analogous Q composite is its conditional closure. Induction on formulas proves both complete formula round trips.

4.3 Arithmetic compilation and both numerical composites

Each fixed scalar update is elaborated as a function symbol. There is no variable ranging over function codes and no uniform object-language evaluator. In one direction, iPRA defines a fixed iterator by primitive recursion whose update ignores its recursion counter. Its equations and every literal-equation induction instance are then available.

For the reverse direction, the inherited internal scalar proofs proceed in the following order:

1. Construct a scalar predecessor P and prove $P0 = 0$ and $P(Sn) = n$. This uses equation induction and numerical cases, without assuming pairing, equality reflection, or successor injectivity.
2. Define truncated subtraction by iteration of P and construct a Boolean characteristic term for *actual* equality. Prove the reflection equivalence throughout the numerical domain.
3. Translate every fixed quantifier-free formula into its characteristic equation. Contextual equation induction therefore supplies every quantifier-free induction instance of iPRA.
4. Construct pairing and projections by bounded search, and compile every fixed primitive-recursive definition into scalar iteration.

For example, with pairing π and projections L, H already proved, a primitive recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p})$$

is represented by

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))),$$
$$f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; projection equations then give both defining equations for f^*. In $\mathcal{F}$, use the normal form of this finite term. It has the same proved defining equations by the normalization comparison.

Uniqueness is internal, not an assertion about standard natural numbers. For two iterators with the same initializer and pointwise equal updates, induct on equality of their values. For ordinary primitive recursion, use the same argument with its counter-dependent update. External induction on a finite function description shows that its double translation satisfies its original equations and hence is provably equal to the original function.

Both numerical interpretations have domain formula $x = x$ and retain actual equality. Consequently the identity graph $x = y$ is internally total, functional, bijective, and preserves each interpreted operation. Combined with the formula composites, this proves bi-interpretability. Finite proof translations preserve $0 = S0$ and ordinary falsity in both directions, giving equiconsistency. This does not assert that either theory proves its own consistency.

Status of the inherited formalization. The pinned repository supplies Lean statements for native/iPRA faithfulness, both formula round trips, and the contradiction equivalence. Those source endpoints were inspected; the upstream development was not rebuilt here. The new full-normal-form calculus, its normalization metatheory, and the Python correspondence have written proofs and executable checks, but no new Lean or Rocq verification is claimed.

5 Implementation, evidence, and remaining limits

5.1 The exact trusted boundary

The selected checker is `goal_checker/code/full_expression_checker.py`. The readable block-form variant is `goal_checker/code/full_checker.py`. Each is a standalone Python module importing only `json` and `sys`. No proof translator, external theorem database, generated table, local helper, or previously checked proof is imported.

The input is an object with exactly four keys: `variables`, `context`, `goal`, and `proof`. The numerical scope is an exact nonnegative integer; Boolean indices are rejected. Terms and formulas are tagged finite lists, with de Bruijn indices. An iterator adds one bound variable in its update only; Q adds one and D adds two in their formula bodies.

One recursive traversal checks constructor arities and child sorts, performs variable actions, and normalizes exposed computations. Validation calls it with the scoped identity action and requires its output to equal the supplied syntax. Thus all supplied used terms are canonical. The traversal checks *all original children* before erasing a computation, preventing an invalid index or wrong sort from being hidden in a discarded body.

The checker validates a current goal before closing it as a retained hypothesis or a reflexive equation. Those automatic leaves ignore their proof payload; `null` is the conventional spelling. This is an intentional input convention shared by the matched controls, not a claim that unused JSON data have passed a separate proof-tree grammar. All retained context formulas are validated, including unused context entries. Non-leaf packets implement Tr, QE, DE, or WI, checking every used major and parameter before substitution.

Success means exit status zero and exactly `ACCEPT` followed by a newline. Every exception or exhausted resource is rejection, not acceptance. Mathematical soundness is induction on the finite accepting call tree; completeness is induction on a derivation, supplying its intermediate major formulas in the packets. The numerical interpretation is the ideal unbounded finite-tree specification. Physical recursion and memory limits can reject otherwise valid very large certificates.

5.2 Reproducible measurements

Each complete source is parsed with CPython 3.12.14 and measured after

```
ast.unparse(ast.parse(source)) + '\n'.
```

No AST nodes, docstrings, validation, normalization, or command-line acceptance code are removed. Significant tokens exclude only comments and structural whitespace tokens. AST counts include every node returned by `ast.walk`. Deterministic gzip uses level 9 and timestamp zero; the recorded zlib runtime is 1.3.2.

The reported comparison is reproducible with

```
python3 goal_checker/reproduce.py
python3 forward/verify.py
```

The first script regenerates the four whole-module measurements, normalized sources, gzip streams, and its checker receipt. The second regenerates the forward translation and normalization tests. Neither script is part of a proof kernel. Other Python or zlib versions may change normalized formatting or compressed lengths.

The large difference between the earlier 3095-byte forward source and the 2529-byte selected checker is 566 bytes, about 18.29%. The packet format and validation organization changed, so this is a whole-implementation comparison, not an isolated axiom experiment. The matched table in Section 1 is the appropriate test of the incremental full-normalization change.

5.3 Executed checks

The goal-directed family passed 680 command-line executions, half in ordinary Python and half with `-O`: 440 expected acceptances and 240 expected rejections, with no mismatches. The 166 distinct fixtures include every primitive rule, open contexts, nested binders, scope escapes, Boolean indices, wrong tags and arities, invalid data inside erased redexes, and bad inference endpoints. Additional independent-forward comparisons passed 500 normalization checks, 500 lifting checks, 250 substitution checks, and 500 canonical-validation checks.

The portable forward package translates all 64 packaged zero-normal source certificates. It passed 132 command-line checks, 1000 normalization-idempotence checks, 1000 substitution-coherence checks, 1000 independent numerical evaluation comparisons, and 2000 defining-equation checks. These are finite implementation tests. They supplement the universal written proofs and are not a proof-assistant certificate.

One regression is particularly informative. The formula

$$D_{x,y}(y = 1)$$

is intuitionistically true under its reading: its conditional antecedent is $0 = 1$, from which every $y = 1$ follows. A raw atomic *case* introduction can prove it using a nonliteral zero witness. That witness becomes literal 0 after full normalization, where the same unmodified packet would instead ask for progress from $z = 1$ to $Sz = 1$. The naive translated certificate is rejected. The compound-envelope translation produces an accepted certificate for exactly the desired closed formula. This test exercises a real introduction-mode mismatch.

5.4 The external normalization cost

Normalization is not a free oracle. Its executable recursion is part of the kernel and its mathematical termination proof is part of this report. It can nevertheless perform very large computations and produce very large terms. The uniform normalizer is not primitive recursive, as proved in Section 3. No primitive-recursive bound on this algorithm, and no internal iPRA proof of its uniform totality, is asserted.

If one additionally requires a manifestly primitive-recursive certificate relation, use explicit bounded computation traces or the raw native calculus. A trace checker would be a different measured implementation; its code is not silently omitted from the present counts. The current user specification concerns the size of a straightforward proof checker, so a total recursive normalizing implementation is a legitimate candidate, with this cost made explicit.

5.5 What remains unresolved

The result does not prove a lower bound on the size of any iPRA checker, nor a universally simplest logical vocabulary. Deleting an explicit computation axiom while implementing its normalization is a change of proof presentation, not an independence result showing that arithmetic computation has ceased to be necessary. Reflexivity still seeds equality reasoning, and the displayed mathematical inventory has six patterns even though the program checks two of them as automatic leaves.

For the requested complexity objective the evidence is mixed: the selected matched source is shorter by three bytes, while tokens, AST nodes, and gzip length increase. The stronger result is the complete normalization and interpretation theorem, including the compound-envelope repair. A substantial reduction below all current checker profiles remains open.

Attribution and the inherited arithmetic foundations

The five sections that follow reproduce the mathematical source files in `Research/certified_zero/reference/` of the pinned repository snapshot. They are included so that the scalar arithmetic and its full first-order interpretation bridge can be checked without fetching external material. Their wording, including phrases such as *new result* or *this paper*, belongs to those earlier reports, not to the novelty claim of the present construction. The corresponding local files are byte-preserved, and `PROVENANCE.json` records their hashes and exact source URLs. The source repository's MIT No Attribution license is included.

The present contribution is full normalization of *both* computations, the compound-envelope repair, the resulting equivalence theorem, and the complete matched implementations. The predecessor construction, reflection of equality, primitive-recursion compiler, logical macros, and original native/iPRA bridge are inherited and explicitly credited [3, 4, 5, 6, 7].

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \;\; \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F} \;\; \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \;\; \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)} \quad \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t, 0), [B(t, z)]_{\mathrm{at}} \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}. \tag{Old DI}$$

The bracket supplies $B(t, z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x, y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t, 0), E(t, z) \vdash E(t, Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t, Sz)$ under $\Gamma, E(t, 0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t, y)$ and all retained data. Under $\Gamma, E(t, 0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t, y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate

the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [5] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{3}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{4}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{5}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{6}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{7}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (7) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{8}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (7); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{9}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (8), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{10}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{11}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (11). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (12) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (13) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (14)$$

The base is (13). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (15)$$

The first uses the comparator recurrence. The second uses (14) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (16)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (17)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (16) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [6]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n,a,b) = I(n,a;u.b)$, addition is $x+y = I(y,x;u.Su)$, and truncated subtraction is $d(x,y) = I(y,x;u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{18}$$

$$C(0,a,b) = a, \qquad C(Sx,a,b) = b. \tag{19}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{20}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{21}$$

and induction for each *literal equation* $E(n,\vec{p})$:

$$\forall \vec{p}\,\left[(E(0,\vec{p}) \wedge \forall n(E(n,\vec{p}) \longrightarrow E(Sn,\vec{p}))) \longrightarrow \forall n\, E(n,\vec{p})\right]. \tag{22}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (22) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{23}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{24}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{25}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (18). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (24); symmetry and (21) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{26}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{27}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{28}$$

To prove the range assertion, split (20). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (19) also give, for all u, v,

$$C(u, v, v) = v. \tag{29}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{30}$$

If $u = 1 = S0$, (25) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{31}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{32}$$

Using these equations and (22), we now derive

$$0 + y = y, \tag{33}$$
$$(Sx) + y = S(x + y), \tag{34}$$
$$d(Sx, Sy) = d(x, y), \tag{35}$$
$$d(Sa, a) = 1. \tag{36}$$

Each induction below has a single equation as its induction formula.

For (33), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (34), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (34) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{37}$$

Indeed, both sides are $S(y + r)$.

For (35), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (36). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (35) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{38}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (33) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (29).

For the step, assume (38) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (38) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (37) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (38). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{39}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (38). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (36) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{40}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{41}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{42}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (27), which is equivalent to $d(Sa, b) = 1$ by (30), and then to $a = b$ by (39). The range assertion is (26) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (41) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{43}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{44}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{45}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{46}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{47}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{48}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (41)–(42). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (48) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (48), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [7]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v + 1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$
$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

F The selected complete checker

This is the entire selected command-line kernel. It imports only the Python standard library. Comments are present for review; all executable code and normalization are included in the measurements.

```
# Complete standalone goal-directed full_expression normal-form checker.
# See ../FORMAT.md for syntax, proof packets, and the exact acceptance contract.
# No helper module is imported; exceptions reject; success is exactly ACCEPT.
import json
import sys


def require(condition):
    if not condition:
        raise ValueError('Invalid certificate')


def walk(a, f, d=0, sort=-1):
    require(type(a) is list and a)
    tag, *args = a
    require(sort < 0 or (tag in ('=', 'q', 'd')) == sort)
    if tag == 'v':
        require(len(args) == 1)
        return f(args[0], d)
    binders = {'z': [], 's': [0], 'i': [0, 0, 1], '=': [0, 0], 'q': [1], 'd': [2]}[tag]
    require(len(args) == len(binders))
    a = [tag] + [walk(b, f, d + k, tag in ('q', 'd')) for b, k in zip(args, binders)]
    return a[2] if a[:2] == ['i', ['z']] else sub(a[3], [tag, a[1][1], *a[2:]]) if tag == 'i' and a
    [1][0] == 's' else a


def syntax(a, n, sort=0):
    def variable(i, d):
        require(type(i) is int and 0 <= i < n + d)
        return ['v', i]
    require(walk(a, variable, sort=sort) == a)


def shift(a, k=1):
    return walk(a, lambda i, d: ['v', i + k if i >= d else i])


def sub(a, t, drop=1):
    return walk(a, lambda i, d: ['v', i] if i < d else shift(t, d) if i == d else ['v', i - drop])


def prove(a, p, g, n):
    syntax(a, n, 1)
    if a in g or a[0] == '=' and a[1] == a[2]:
        return
    match p:
        case ['qe', b, t, p, q]:
            syntax(t, n)
            prove(['q', b], p, g, n)
            require(a == sub(b, t))
            prove(sub(b, ['z']), q, g, n)
        case ['de', b, p, q]:
            prove(['d', b], p, g, n)
            prove(shift(a), q, [['q', b]] + [shift(x) for x in g], n + 1)
        case ['di', t, p]:
```

```
            require(a[0] == 'd')
            syntax(t, n)
            b = sub(['q', a[1]], t)[1]
            h = b if b[0] == '=' and t == ['z'] else shift(sub(b, ['z']))
            prove(sub(b, ['s', ['v', 0]], 0), p, [h] + [shift(x) for x in g], n + 1)
        case ['tr', s, t, h, p, q]:
            prove(['=', s, t], p, g, n)
            syntax(h, n + 1)
            require(a[0] == '=' and a[2] == sub(h, t))
            prove(['=', sub(h, s), a[1]], q, g, n)
        case _:
            raise ValueError('Invalid proof')


def check(c):
    require(type(c) is dict and set(c) == {'variables', 'context', 'goal', 'proof'})
    n, g, a, p = (c[k] for k in ['variables', 'context', 'goal', 'proof'])
    require(type(n) is int and n >= 0 and type(g) is list)
    for b in g:
        syntax(b, n, 1)
    prove(a, p, g, n)


if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except (ValueError, TypeError, IndexError, KeyError, RecursionError):
        sys.exit(1)
    print('ACCEPT')
```

References

[1] Vladimir Reshetnikov et al., *Concise presentations of intuitionistic primitive recursive arithmetic*. ProveIt, snapshot `11be5aabc6b07262046e2553419bc6b05271d178`, `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md`. https://github.com/VladimirReshetnikov/ProveIt/tree/11be5aabc6b07262046e2553419bc6b05271d178/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt, *Native scalar calculus and its interpretations*, `Lean/ConciseIPRA/Native.lean`, `NativeInherited.lean`, `NativeMinimal.lean`, and `NativeBiInterpretation.lean`, at the snapshot above.

[3] ProveIt, *Complementary introductions with one local assumption*, `Research/certified_zero/reference/inherited_choice_basis.tex`. Reproduced in this report.

[4] ProveIt, *Arithmetic strength and both ordinary interpretation composites*, `Research/certified_zero/reference/inherited_ordinary_bridge.tex`. Reproduced in this report.

[5] ProveIt, *Internal scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`. Reproduced in this report.

[6] ProveIt, *Equality reflection and quantifier-free induction*, `Research/certified_zero/reference/equality_reflection.tex`. Reproduced in this report.

[7] ProveIt, *Scalar compilation of primitive recursion*, `Research/certified_zero/reference/primitive_recursion.tex`. Reproduced in this report.

[8] *One computation axiom for iPRA*, `docs/incoming/ipra_zero_normal_2026-11-11.zip`. The delivered `code/zero_six.py` and intermediate certificates are included as frozen comparison material in `forward/reference/zero_normal/`.

[9] *Canonical recursors*, `Research/canonical_recursors/`. Its canonical successor-seeded recursor calculus and its separate 516-byte compact scalar codec have different input representations.

[10] The other incoming archives at the pinned snapshot: `ipra_diagonal_guard_2026-11-11.zip`, `ipra_initializer_equality_2026-11-11.zip`, `ipra_scope_kernel_2026-11-11.zip`, `ipra_target_closure_2026-11-11.zip`, and `ipra_template_free.zip`. Their main specifications and reported claim boundaries were inspected to distinguish the present result from existing reductions.